

Deadlocks

- Precedence Graph
- Deadlock while Lock Upgrade
- Read for Update

Transactional Synchronization Extensions

From Wikipedia, the free encyclopedia

Transactional Synchronization Extensions (TSX-NI) is an extension to the [x86 instruction set architecture](#) (ISA) that adds hardware [transactional memory](#) support, speeding up execution of multi-threaded software through lock elision. According to different benchmarks, TSX can provide around 40% faster applications execution in specific workloads, and 4–5 times more database [transactions per second](#) (TPS).^{[1][2][3][4]}

TSX was documented by [Intel](#) in February 2012, and debuted in June 2013 on selected Intel [microprocessors](#) based on the [Haswell](#) microarchitecture.^{[5][6][7]} Haswell processors below 45xx as well as R-series and K-series (with unlocked multiplier) [SKUs](#) do not support TSX.^[8] In August 2014, Intel announced a bug in the TSX implementation on current steppings of Haswell. Haswell-E. Haswell-EP and early [Broadwell](#) CPUs. which resulted

Recap: Conflicts in a schedule

- Conflicts are a concept to capture the pairwise relationship of operations (and by extension transactions) accessing the same object in a schedule.
- Each operation writing an object has one conflict with each operation by a different transaction accessing this object.

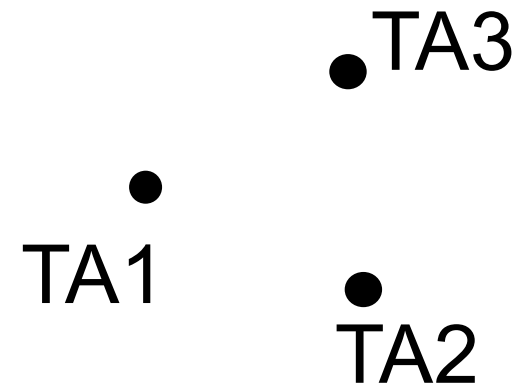
We write these conflicts for object x :

$r_1[x], r_2[x], c_2, w_1[x], c_1, r_3[x], w_3[x], c_3$

- As ordered pairs of operations:
- $r_2[x], w_1[x]$, and $w_1[x], r_3[x]$, and also $w_1[x], w_3[x]$,
- $r_1[x], w_3[x]$, and $r_2[x], w_3[x]$.
- Idea: order in which transactions commit should be compatible with this order.

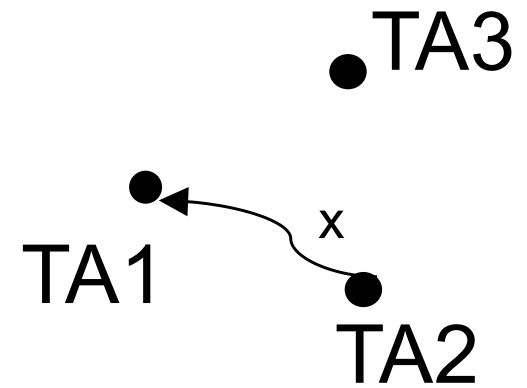
Precedence graph

- We interpret the conflicts as directed edges, which we see as going from the transaction of the earlier operation in the conflict to the transaction of the later operation.
- The resulting directed graph we call the **precedence graph**, of that schedule.
- Idea still: order in which transactions commit should be compatible with this order.
- Definition: A schedule is conflict-serializable iff the precedence graph is free of cycles.
- $r_1[x], r_2[x], c_2, w_1[x], c_1, r_3[x], w_3[x], c_3$



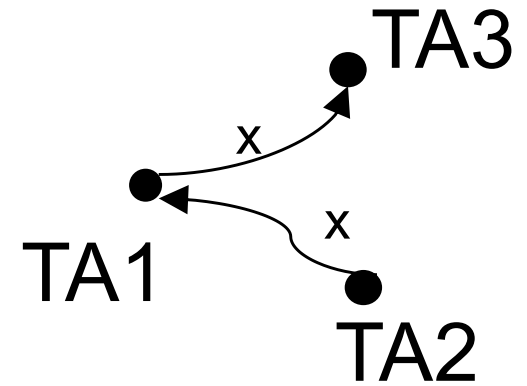
Precedence graph

- We interpret the conflicts as directed edges, which we see as going from the transaction of the earlier operation in the conflict to the transaction of the later operation.
- The resulting directed graph we call the **precedence graph**, of that schedule.
- Idea still: order in which transactions commit should be compatible with this order.
- Only if the precedence graph is free of cycles the schedule is serializable.
- $r_1[x], r_2[x], c_2, w_1[x], c_1, r_3[x], w_3[x], c_3$



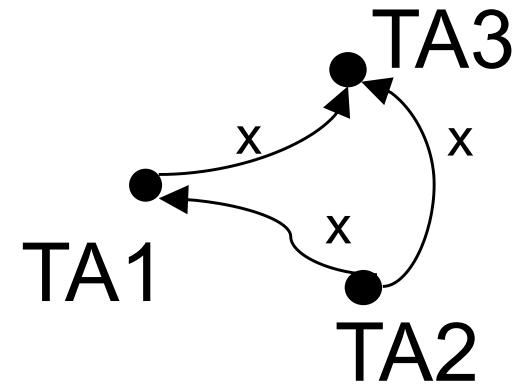
Precedence graph

- We interpret the conflicts as directed edges, which we see as going from the transaction of the earlier operation in the conflict to the transaction of the later operation.
- The resulting directed graph we call the **precedence graph**, of that schedule.
- Idea still: order in which transactions commit should be compatible with this order.
- Only if the precedence graph is free of cycles the schedule is serializable.
- $r_1[x]$, $r_2[x]$, c_2 , $w_1[x]$, c_1 , $r_3[x]$, $w_3[x]$, c_3



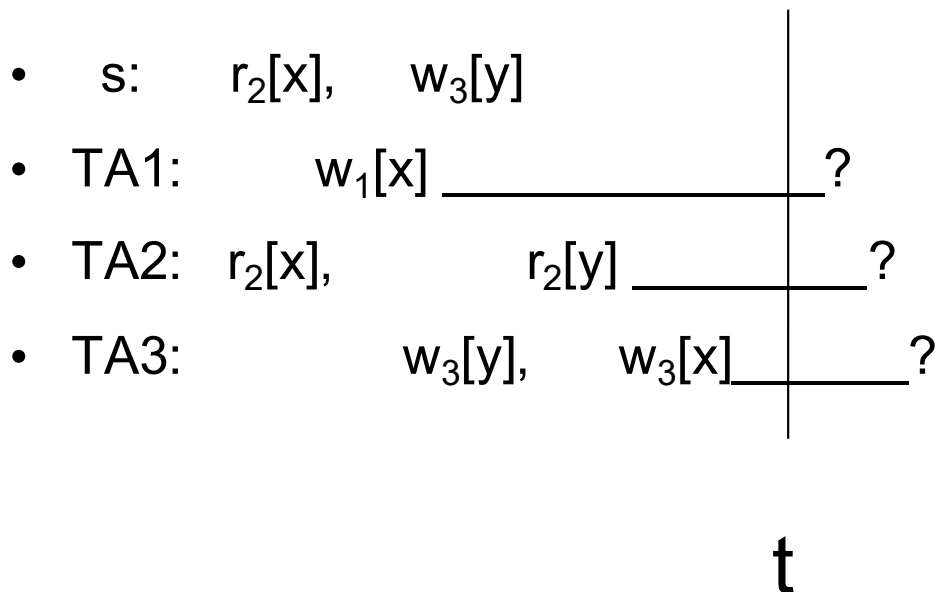
Precedence graph

- We interpret the conflicts as directed edges, which we see as going from the transaction of the earlier operation in the conflict to the transaction of the later operation.
- The resulting directed graph we call the **precedence graph**, of that schedule.
- Idea still: order in which transactions commit should be compatible with this order.
- Only if the precedence graph is free of cycles the schedule is serializable.
- $r_1[x]$, $r_2[x]$, c_2 , $w_1[x]$, c_1 , $r_3[x]$, $w_3[x]$, c_3



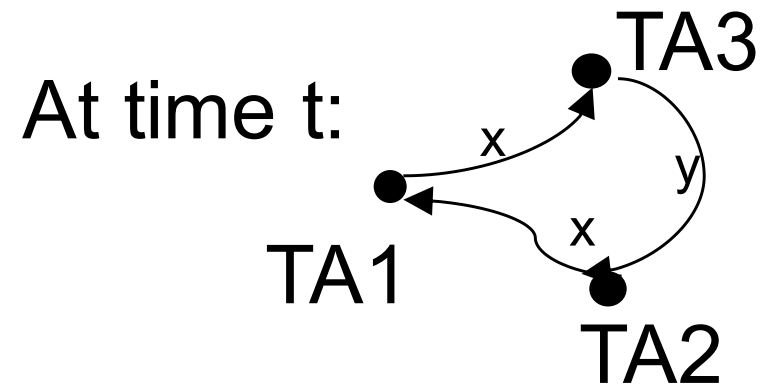
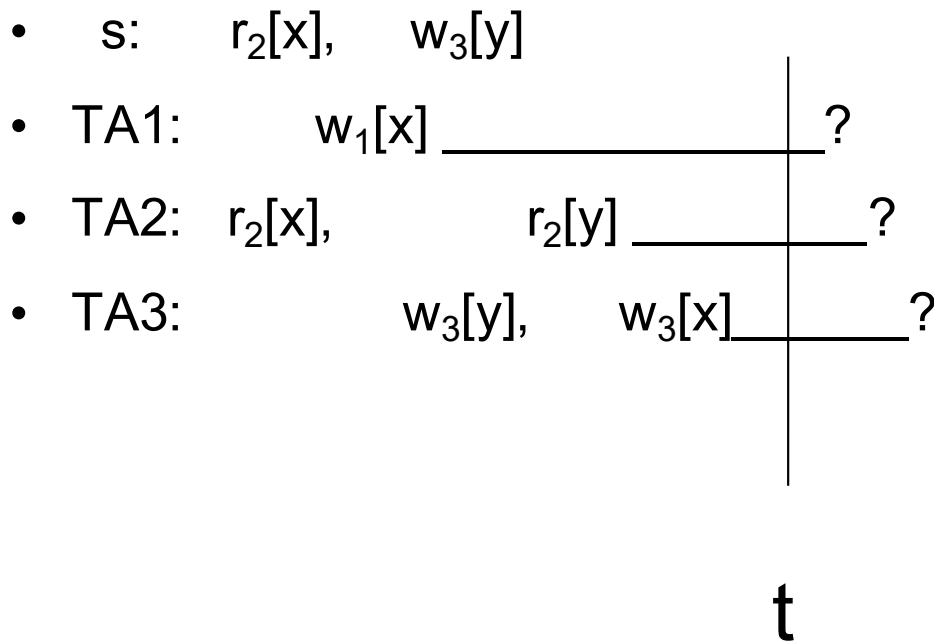
Deadlock: cycle in precedence graph

- Deadlock:
- The precedence graph including the operations blocked and not yet executed at a point t in time contains a cycle:



Deadlock: cycle in precedence graph

- Deadlock:
- The precedence graph including the operations blocked and not yet executed at a point t in time contains a cycle:



Deadlocks while attempting lock upgrade

- Transactions mostly read x before they write x .
- On the common scheduler, this results in a lock upgrade: the transaction first has a read lock on x , then upgrades this to a write lock on x .
- This process can result in a *deadlock*:
 - s : $r_1[x], r_2[x], \dots \quad ?$
 - TA1: $r_1[x], \quad w_1[x] \quad \underline{\hspace{2cm}} ?$
 - TA2: $r_2[x], \quad w_2[x] \quad \underline{\hspace{2cm}} ?$
- This is bound to happen eventually, if transactions take some finite amount of time.
- Solution: Clients declare intention to write....

Early declaration of intention to upgrade

- The aforementioned *deadlock*:
 - s: $r_1[x], r_2[x], \dots$?
 - TA1: $r_1[x], w_1[x]$ _____?
 - TA2: $r_2[x], w_2[x]$ _____?
- Solution: Use “SELECT ... FOR UPDATE”
- This is a read operation that expresses intent to write.
- We indicate this operation with capital R[] in the schedule (this is an extension of the basic transaction model):
- Implementation is left to database, we use:
- **R[] is, with regard to locking, treated like a write:**
 - requires X lock (and U lock) before even reading.

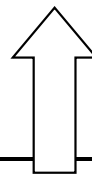
Early declaration of intention to upgrade

- The aforementioned *deadlock*:

- s: $r_1[x], r_2[x], \dots \quad ?$
- TA1: $r_1[x], w_1[x] \quad \underline{\hspace{2cm}} ?$
- TA2: $r_2[x], w_2[x] \quad \underline{\hspace{2cm}} ?$

- Alternative using R (“SELECT ... FOR UPDATE”)

- s: $R_1[x], w_1[x], c_1 \quad R_2[x], w_2[x], c_2$
- TA1: $R_1[x], w_1[x], c_1$
- TA2: $R_2[x] \quad \underline{\hspace{2cm}}, w_2[x], c_2$



transaction TA2 is waiting, because TA1 has the exclusive lock on x

- the simple scheduler: all reads are “FOR UPDATE”

Only one transaction uses R[]

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

Only one transaction uses R[]

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- We have to address all possible timings (either by individual testing or by argument)
- First timing
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- second timing
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- And so on.....

Only one transaction uses R[]

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- **No.** We have to address all possible timings (either by individual testing or by argument)
- First relevant timing: TA2 can only access x after c_1
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- Second relevant timing: TA1 can only access x after c_2
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

Only one transaction uses R[]

- Can the following transactions run into a deadlock ?
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2
- **No.** We have to address all possible timings (either by individual testing or by argument)
- First
 - s1: $r_1[x]$, $w_1[x]$, c_1 , $R_2[x]$, $w_2[x]$, c_2
 - TA1: $r_1[x]$, $w_1[x]$, c_1
 - TA2: $R_2[x]$ _____, $w_2[x]$, c_2
- 2nd
 - s2: $R_2[x]$, $w_2[x]$, c_2 , $r_1[x]$, $w_1[x]$, c_1
 - TA1: $r_1[x]$ _____, $w_1[x]$, c_1
 - TA2: $R_2[x]$, $w_2[x]$, c_2

Summary

- Conflicts lead to the precedence graph:
- conflicts seen as directed edges between transactions.
- Circles in the precedence graph including blocked operations express deadlocks
- Shared reads are prone to deadlocks without precautions.
- Certain deadlocks can be prevented with R[].